Meeting @October 16, 2026 10:55 AM

Summary

Action Items

- [ ] Mikhail to start recording and transcription for future meetings ▸ ▸
- [ ] Team to divide core responsibilities (front-end, back-end, database, AI model) in next team meeting ▸ ▸
- [ ] Team to research and evaluate alternative AI models (SentenceBERT, CodeBERT, GraphCodeBERT, JPlag) as replacements or supplements to RoBERTa ▸ ▸ ▸
- [ ] Team to revise SRS document based on professor's feedback and research findings ▸
- [ ] Team to create a system architecture diagram ▸
- [ ] Team to explore RAG-based approach and vector database for file comparison ▸ ▸
- [ ] Team to establish biweekly agile/sprint meetings and testing cadence ▸ ▸

Meeting Context

- Stakeholder meeting between the capstone student team and their professor/stakeholder to review the submitted SRS document and discuss the project direction
- Participants included Mikhail, Vince, and Dennis (additional team members were still being waited on) ▸ ▸
- The professor requested that the meeting be recorded and that Teams generate meeting notes ▸ ▸

Project Overview

- The system is an academic integrity tool similar to Turnitin, designed to detect: ▸
  - **Plagiarism/similarity:** checking student submissions against a database of prior submissions ▸
  - **AI-generated content:** scanning document text for non-human writing patterns ▸ ▸
- The professor (not the system) retains final say on whether a submission violates academic integrity ▸
- Graduating students are completing this as their capstone project; some members are in their final semester ▸ ▸

Professor's Feedback on SRS

- Advised the team to **identify and prioritize core MVP features** that are realistically implementable before attempting everything simultaneously ▸ ▸
- Highlighted that **similarity detection and AI-generated content detection are two separate problems** that need to be clearly distinguished in the architecture ▸
- Noted that **RoBERTa is not inherently an AI-generated text detector** and does not handle programming assignments (which require code-aware models) ▸ ▸
- Suggested additional tools to consider: ▸ ▸ ▸ ▸ ▸
  - **SentenceBERT (SBERT):** semantic similarity between written documents
  - **CodeBERT:** semantic representation of programming code
  - **GraphCodeBERT:** structural comparison of code assignments
  - **JPlag:** useful for programming assignment plagiarism detection
- Recommended considering **lexical** (word-to-word), **structural** (formatting), and **semantic** similarity dimensions ▸
- Professor agreed that SentenceBERT or CodeBERT would be **more reliable than RoBERTa** for the project's core similarity and derivation analysis goals ▸

Proposed Technical Architecture

- **Databases:** At minimum 2 MySQL databases — one for user/admin authentication and one for student assignment storage ▸ ▸
- **AI detection approach discussed by team member:** ▸ ▸
  - Use a **RAG (Retrieval-Augmented Generation) pipeline** to process uploaded files
  - Convert documents into **vector embeddings**, stored in a **vector database**
  - Compare vectors across submissions to detect similarity/plagiarism
  - Reserve **model training for AI-generated text detection** as a future (MVP 2) enhancement ▸ ▸ ▸
- Final database structure anticipated: ▸ ▸
  - **MySQL** for authentication and user data
  - **Vector database** for file embeddings and comparison
- Professor responded positively to the RAG approach and asked if the team would be using **LangChain** ▸

Testing & Development Process

- Planned approach: **agile/sprint meetings** roughly weekly to track progress ▸
- Minimum testing cadence: **biweekly**, with more frequent testing as features are added ▸
- Professor suggested adding **instructor verification** as a mechanism to validate accuracy and reliability of similarity results ▸
- Team acknowledged that the **AI implementation is the most challenging component** and plans to assign multiple team members to that work ▸ ▸

Notes

Transcript

No, suffer.

Hello?

Hello?

It's all mine.

Do you hear me?

Hello?

Yo, can you hear me?

Oh, hold on one second. I can't hear you.

Let me see if mine— yeah, it's working on my side.

One second.

Uh, hello?

You?

Yeah, my stuff is still acting weird.

Can you hear me?

Um.

Hello?

Still nothing. One sec.

Yeah, but I, I can't hear you, so One moment. They're gonna— oh, there we go.

That's weird.

Okay, now it's acceptable.

Can you hear anything?

Yeah, I can hear you now.

All right, but what's up, bro?

Yeah.

Me, low-key.

My camera is supposed to be working, but Oh my goodness, it's having issues right now.

Okay, it's straight. Should be okay.

Um, cobstone.

Okay.

Whoa, okay.

Yeah, let me see. Hold on.

Thank.

No, there you go.

Yo, yeah, I see you now.

All right, um.

And.

You know who the stakeholder are supposed to be?

His name is

Meeting with is like their name.

It's like here, I'll drop it. I don't know how to pronounce it, honestly.

Okay.

Where is it? Yeah.

I'll just put it in the chat real quick.

It's like a— there it goes.

A rug.

Yeah, seems like it.

Hmm.

But yeah, I don't know what we're gonna talk about, honestly. Like, if he's just gonna grill us about the project or, or what.

That's what I imagine.

Yeah.

Just them.

Hmm.

Are you on Mac?

Yeah.

Okay.

Yeah, I saw you staring like directly at the screen and then a text message summoned in the group chat.

The link is invalid.

Bro.

You graduate, um, This fall or in spring?

Uh, in spring.

Oh well, makes sense, capstone.

Yeah.

Got this and then a couple more classes and I'm done. Is this Stamps.

What about you?

Ah, same capstone. So, uh, yeah, I gotta be here for another semester.

Good morning.

Still waiting on a couple of people, but if they don't join, it's, it's fine. We can discuss about the project with you.

Yeah.

Sure.

Let's see.

As you're the organizer for the meeting, can you make sure that Teams will generate the notes, meeting notes?

Yes, yes.

Let me— I can go ahead and record it and start transcribing.

Yeah.

Yeah.

And if it is possible, if you can record the meeting.

Yeah.

All right, so recording and transcribing is currently, uh, on right now.

Okay.

Hey, good morning.

Morning, guys.

We're just waiting on one more person, but by 11:05 we can just get started.

Okay.

All right, we can— we're good to go ahead and start, Professor.

Please, you can start.

All right, so I had emailed you the SRS document. Did you have a chance to take a look at it and see pretty much the required areas that we had for the project?

Okay.

Yeah, I did that. Do you have any kind of presentation for today's meeting prepared? Any kind of like

Um, I didn't really.

I didn't really have like an agenda of what to speak about. Pretty much I was just going to go over like a high-level overview of what it is, why it's needed. Um, the tech stack, the functionality, and so on and so forth. And if you had questions or concerns or any kind of, um, suggestions that could help us along the way that we may have not thought of, or we, you know, it may not be like in the current MVP for the first class for Capstone.

That's something that we would also be willing to hear and add to like a list for, uh, for the future, um, like MVP 2 or anything like that.

Okay.

But

Yeah, so first, can you explain the whole workflow, like right from the time when the submission happens until it gets evaluated by the system? So can you Just briefly explain. Any one of you can just briefly go through it.

Yep, I mean, I can pretty much go through it. So if we're speaking on the, the user end, pretty much this is going to relate similarly to Turnitin, which is just another, uh, plagiarism checker or Um, in our case, it's going to be a little bit more than that, but what it would do is it will check the student submission for any like similarity or any kind of signs if something was copied, paraphrased, or even AI generated and then it would flag those specific portions for the professor to go

ahead and review it, review it, and then the professor would have ultimately the final say whether this was fully done on the student's end or if this was generated via AI or, uh, copied or anything like that, as opposed to deciding on its own.

And so that, that would pretty much be the, the overall like front to end of what would happen on the, like the real-life example. You would turn in an assignment, the, the

Okay.

Functionality behind it would essentially be able to pair or go through and scan the entire document, check for anything that's AI generated, anything that's been paraphrased from other student submissions as well that have also gone through the system and then it would flag it for the professor to review. So that would pretty much be like the front-to-end, like, real-life example. However, like, breaking it down into like The functionality of, okay, what is checking it, and then like you know, how it's able to know if something is AI or not, or if something was paraphrased.

That's a little bit more in-depth. And, um,

Okay.

I can speak on it, Vince. You— would you want to speak on like the functionality of it or, or anything like that? Yes, Professor.

Uh.

No, I have, I have went through your— yeah, I have went through your SRS document. Then I have few suggestions to make with. The first one is you first try to identify the core functionality you can realistically implement and demonstrate.

Okay, so once you do that, then you can attempt everything simultaneously. So like prioritizing reliability, reliable similarity analysis system, and then you introduce the advanced feature. So rather than focusing on all of them, just focus on minimum viable product which we can develop. So that was one of the suggestions. And other thing is about how are you going to distinguish between similarity detection and AI-generated content detection, because these are 2 separate problems.

Yes, correct.

Yeah, um, so pretty much within the backend, so we would have a database that stores user logins. Um, but we would also have an AI model, Roberto, which pretty much scans for the AI-generated writing as well. So we would have our, um, own pretty much like LLM or, uh, open source LLM within our backend system that would be able to scan for that AI writing, whereas paraphrasing, we wouldn't essentially need to use that.

We would actually have a separate database where all of these students' assignments are stored, in which the AI model would then check the differences in between those 2 databases, but it wouldn't actually scan the document.

Right.

And so that would be the difference. Um, however, our AI model would be able to — our AI model is the checker.

Um, it checks not only for, um, within the, within the 2 different databases that we have for, uh, similarities within students' assignments. Um, for the plagiarism and the copying portion, but when it comes to, um, AI-generated documents or code or anything like that, the, um, AI would actually scan the document itself and try to detect, um, just like Uh, wording, uh, like, like verbiage that just isn't human.

Um, and that would pretty much, uh, be how it would, uh, scan the, the differences or how it would be able to know if something was AI generated or not.

Okay.

Okay, and do you need any kind of databases?

Yes, so we're going to use a MySQL database. We're going to use one for user logins because I, I believe, um, We had functionality where we would like to have user and admin logins, and so all of those login information would have to be stored in its own database. And then for assignments, we would have its own database where whenever a student uploads something, that assignment would then be placed in that specific database.

So at minimum, we would need 2. So that would be the MVP for the first round. So we would need 2 databases, but And as, as we, as we proceed, I'm sure there could be a question of maybe adding another one to make something a lot more smoother, but at minimum it would be 2.

No.

Mm-hmm.

MySQL databases.

Okay.

Okay, and how are we going to test, test the system? So means like determining, suppose, whether similarity results are accurate and reliable.

So I believe testing would need to happen So I would assume— we haven't really spoken about this yet, but I would assume that we would have agile and sprint meetings pretty much every week to discuss where we're at in the project, and then let's say if we decide that we're at a testing phase where we have implemented our databases, when a— when something as important like a database has been implemented, we want to make sure that it's working properly so that we don't continue with something that we haven't tested.

So I would, I would probably say we haven't had the conversation, but we will. I would probably say biweekly testing, especially if we've implemented you know, a backend system, or if we're doing something on the frontend that we need to check. And so that would be at minimum is a biweekly testing period, but I would say testing could happen a lot more frequently just depending on what we add to the project itself.

Okay, good. And I think we can add some kind of instructor verification as well in this.

Okay.

To determine whether the similarity results are accurate and reliable, right? And then based on the current, based on the current SRS you have shared, What do you believe, like which parts of the SRS are complete and which sections we still need clarification or revision on that?

Okay.

Um, Vince, I know you and Dennis were speaking about it. Um, was the Roberta, like the AI portion, something that we were like, okay, this might be a little tricky, or how was— how's when you were looking at like the functional requirements and, and speaking to Dennis about that, were there any concerns or anything that we would want to bring up by chance or no?

Um,

Not really, but I do feel like there's a couple of things missing in terms of functional requirements, and I want to go through the sheet again to make sure we can touch on everything.

Okay.

Let me think.

Uh, yeah, no, I can't, I can't really think of anything that we could.

Quick touch on, but

Okay.

Yeah, no, I can't.

Okay.

On my end,

Yeah, because, yeah, yeah, because, uh, one thing is there that Robert A, which you have decided, it is basically not an inherently AI-generated text detector, right?

Yep. Mm-hmm.

So we need to— and then how are we going to handle the programming assignments? Because Robert, I guess, only works with natural language text, doesn't work with, uh, I mean,

Yeah.

So we haven't, we haven't divided the core responsibilities within our team. That will probably be one of our next meetings that we'll have to discuss.

Okay.

Okay.

will be in charge of what. So, you know, we're gonna create a web application pretty much. So We know that we're gonna want— we're gonna have to have a front-end developer and a back-end developer and database developer as well. And there's going to be responsibilities that will need to be distributed out in which whoever is in task of Roberta as well, I believe, won't do it by themselves since that is a lot more responsibility and a lot more research that has to go into it.

But I will say

Got.

I definitely hear your, your concerns with it and how it can be a lot more challenging. And so what I, what I can say at the moment is that With knowing this, we'll put a lot more emphasis on, on focusing on that portion just because it can be a little bit more difficult. And I believe that is the most challenging portion of our project is the AI implementation, just because if we didn't have this, then it would just be checking a document against another document in a database, and that's fair— fairly simple.

So the AI portion that scans the actual writing of the documents is the hardest portion, and so At this moment, I believe if we do understand that, we can just put a lot more bodies on that specific part of the project so that You know, that's if it was just one person working on it, they wouldn't be confused all the way or just be lost. So I think, um, just like being able to like work as a team on that would, would, uh, at least help.

And then again, um, if this is something that we feel as if

Let's, let's say we do more research and we feel as if there's another way that may be better and a little bit easier. That's something that we can either speak to you or even about the professor about to see if it would be best to implement that as opposed to Roberta but at the moment, from what we've like researched about it, do believe that may be the initial best thing to start out with.

But then again, we'll just have to see.

Right, so yeah.

Yes.

So my specific thing which about the architecture, it can be like when student uploads the assignment, our system would perform some kind of a content extraction from the assignment, would perform some kind of preprocessing if required, and then we also have a Roberta detector. But at the same time, if we can come up with some other core similarity engine which will look at the lexical thing, structural thing, and semantic thing.

Okay, so how 2 assignments are lexically means word-to-word similarity, then structural similarity like the formatting and all, and then there is some kind of semantic comparison as well. So for that, uh, there are some other models as well. So I remember one asset one sentence BERT is there, SBERT, which looks at

semantic similarity between the written documents. Then regarding programming assignments, we have CodeBERT.

Right, correct.

Yeah, which also looks at semantic representations of the programming code. In case if, uh, the assign— if the student uploaded assignment contains some kind of images, okay, then in that case we can even have— or maybe if you would like to compare the structure of 2 assignments then we have a Graph CodeBERT.

Okay.

So there are a lot of other tools. You can always check it out on web and see what is appropriate for our, uh, There is a JP Lang as well, so JP

Okay.

I appreciate that.

JP Lang. So that is useful for programming assignments.

I think, I think that I appreciate your feedback on that because thinking about it now, having Roberta manage 2 separate databases would kind of just be a little bit too much, whereas having an AI model separately for semantics and the actual paraphrasing and/or copying of documents from students, I think that will be better. So that will be something that I think would be something that we can implement into the first MVP.

And then Towards the second one if there's, if there's any additions that we can add to it. But okay, well, I'll definitely try to see if there's models that we can, that we can add to

to our project as well that can, that can focus on the, um, different, uh, different things that, uh, that aren't AI generated pretty much.

Yeah, yeah, yeah, because our focus is on similarity and derivation analysis, so I think SentenceBERT or CodeBERT would be more reliable than RoBERTa.

Okay. Okay.

Okay, so just give a thought on that, see whatever is feasible, and come up with some kind of document wherein then We can possibly revise if some SRS is required. And then I would also recommend drawing some architecture diagram, system architecture diagram.

Okay.

Can come up with, right? Yeah. Are y'all able to hear me, by the way, or no?

Yeah, we can hear you.

Yeah.

Okay.

So I think the first thing that I would try and implement for this is a RAG tool, like Professor— or what our professor was saying.

Is like looking for semantics in the documents themselves first, and that allows you to separate what is considered plagiarism or what isn't first, and then store that in a specific database. Semantics are stored as embeddings, and that's using a different type of database. In this case, it would be vectors, and that you— we would be able to compare each vector against each other for each page or each file that's submitted after RAGging.

You would be able to tell what is similar or what isn't, and that's typically how you're able to compare what is like considered plagiarized or not. You would have a database storing all the vectors of all the RAG files.

That's been created for, let's say, X assignment at Georgia State, right?

And you would do a comparison of such. If it shows a high confidence level that, hey, this is really, really, really bad, then you would return, okay, saying that this is flagged, correct?

But training a whole model on top of what we're trying to do here would be a lot of work, and I think that's what he's trying to separate. The initial portion that we're probably going to look to do is use a RAG tool and then turn all the files into vectors and then compare such. And then in the future, you can train something to detect it on a whole bunch of different pieces of data.

And instead of looking for a comparison between, what's it called?

Instead of looking for comparison between 2 physical RAG files.

You would train a model to detect In this case, we would be looking for AI-generated text since that's more of a deterministic thing.

Um, You would train it to detect AI-generated text based off of a sample, and then that would return a confidence level off of what it thinks, or the weights think, is

AI-generated text.

Um, again, the first step would be to RAG the files.

Um, and then store those as embeddings that would require a vector database.

Um, we would have a MySQL database likely for authentication.

Um, like Mikhail was saying, but I think there's going to be 2 different types because of the way that files are stored, especially if we're trying to do comparison. So it'll be a vector database for all the files. And then it'll be a MySQL database for authentication and user information.

Is that correct?

Yeah, yeah, this is very nice, uh, thing you said, uh, for a RAG, I guess, uh, you are going to use LangChain, right?

9.